



Databases for non-event data in the Mu3e Experiment

Urs Langenegger

Mu3e Internal Note 01XX

29th June 2026

This note describes the CDB - the database hosting “conditions” (together with “configuration”) data and the RDB (the run database) for the Mu3e experiment. The hardware and system setup of the server `mu3edb0`, hosting the databases, is described (because this machine also serves as a backup for the partsDB server in Heidelberg, a few remarks on that are also included). The design of the CDB and RDB software, the database backends, the API, and the involved workflows are described. All implemented calibrations are documented from a technical point of view (no detector performance analysis is provided).

The global tags are described in detail.

Note: Substantial textual overlap with Mu3e-Note-0099-ConditionsDatabase.



Contents

1	Introduction	3
2	Hardware and System Setup	7
2.1	CDB and RDB	7
2.2	PartsDB	7
3	Software architecture and API	8
3.1	Architecture	8
3.2	HTTP API	9
4	Calibration Software	11
4.1	calAbs	11
4.2	calEventStuffV2	11
4.3	calFileTimeCalibration	11
5	Workflows and tools	13
5.1	Overview	13
5.2	Lists for detector element IDs	13
5.3	New tag with new initial payload(s)	13
5.3.1	Alignment/geometry payload with root alignment input	14
5.3.2	New payloads based on many input files in a directory	14
5.3.3	New payloads from detector input and initial payload	15
5.3.4	Insert new tag with new (initial) payload(s)	15
5.3.5	Upload new tag with payloads to mongoDB	15
5.3.6	Upload new global tag to mongoDB	16
5.3.7	Download GT and tags with payloads from mongoDB	16
6	Changes to the CDB code	17
6.1	mu3eUtil v6.9	17
6.2	mu3eUtil v6.7	17
6.3	mu3eUtil v6.6	17
6.4	mu3eUtil v6.5	17
7	Global Tags	19
7.1	datav6.3=2025V0	22
7.2	datav6.3=2025V1	23
7.3	datav6.5=2025V0	24
7.4	datav6.5=2025V1	25
7.5	datav6.6=2025V0	27
7.6	datav6.6=2025V1	29
7.7	datav6.9=2025V0	31
8	Tags	33
8.1	Alignment tags	33

Document history:

Ver.	Date	Author	Comment
1.0	January 31, 2026	Urs	Initial version



1 Introduction

The data measured by the Mu3e experiment comprises event-data (the data resulting from interactions of charged particles with the sensitive elements of the Mu3e detector), environmental data (for instance, temperature), and operational data (*e.g.*, beam current and magnetic field strength). As first introduced in Ref. [1], a “CDB” has been set up for the Mu3e Experiment to store and provide access to “conditions” data. In addition, the CDB is equipped to also store “configuration” data (*e.g.* versioned pixel mask files).

The event-data recorded in runs with specific detector configurations and conditions (fixed during a run) require configuration and calibration constants with an interval of validity (IOV) that is a priori not coupled to single runs and, indeed, should cover longer periods of multiple runs (stable conditions should be the ideal). Figure 1 illustrates this situation.

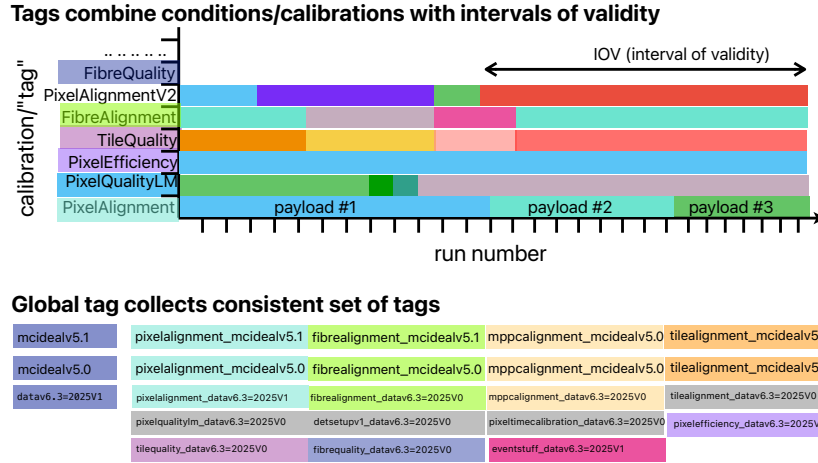


Figure 1: Changing conditions for calibration payloads that need to be managed through the CDB. The time-dependence on the abscissa is indicated through the run numbers, while the labels on the ordinate indicate different calibrations (either different, improved, versions of a calibration, a different type of calibration for the same detector components, or calibrations for different detector components). A payload consists of the calibration constants and descriptive meta-data. Each payload has a specific interval of validity, which is (by design choice) open-ended.

The calibration constants are encoded in “payloads” consisting of descriptive meta-data and and actual data in the form of (eventually¹) a binary large object (BLOB).

By definition, a “tag” is the combination of a name (a specific calibration name plus another identifying string) and a list of IOVs. An example for a “tag”

¹At the moment, the calibration data is not stored in binary form, but rather base64 encoded. This is an implementation detail that could be changed without affecting the CDB design.



describing the alignment of the pixel detector could be for instance

```
tag: pixelefficiency_datav6.3=2025V0 = [1, 100, 200, 5700]
```

where the calibration name is `pixelefficiency`, and the identifying string indicates which `tri-rec` version is required for the usage of this tag together with additional information (version of the alignment payload and that this is for data taken in 2025). The array indicates the runnumbers where the payload changes. Over time a pattern for the identifying string will develop. This identifying string currently is identical to the name of the “global tag” (see below) in which the tag was first used. Often, the term “tag” is used for either the tagname or the compound object (tag name together with the array of runnumbers).

By choice, an IOV is open-ended, *i.e.*, the correct IOV for a given run X is the IOV labeled with the largest run number smaller than X . For example, given a tag with IOV list `[1, 236, 567]`, the correct IOV for run 235 would be IOV 1.

Processing the event-data of a specific run requires the constants for (possibly many) calibrations of all involved detector components, possibly for multiple software versions (in case the data of different data taking periods require different software versions). To manage this complex situation, “global tags” provide the means to organize tags for specific software versions and tasks: A global tag contains a list of tags.

A “hash”² uniquely labels a payload (given a tag with IOV and global tag) in the storage back-end, for example for a “pixel quality” calibration payload of the `mcideal` global tag

```
hash: tag_pixelqualitylm_mcideal_iov_1
```

Here, `tag` and `iov` are fixed components of the hash string, while `pixelquality`, `mcideal`, and `1` indicate variable components (calibration name, global tag, and IOV, respectively) and are different for different calibrations. Fig. 2 shows the CDB browser displaying global tags, tags, and payloads.

The design of the database *storage* is kept very simple and is illustrated in Fig. 3. There are three required collections (or tables, in SQL³ terminology) storing everything related to the conditions data. The three required collections are the “global tags”, the “tags” (or “iovs”), and the “payloads”. In addition, there might be other collections for auxiliary data, *e.g.*, the run database (collecting basic information on run like start and end times, number of events, readout configuration, *etc.*) or meta-data (for easier database access).

The database server `mu3edb0` at PSI hosts the storage and database backends for both CDB and RDB. In addition, it serves as a possible backup server for the `partsDB`.

²At the moment, the hash is a simple string.

³Structured Query Language, often used in the context of relational schema-based databases.



DATABASES FOR NON-EVENT DATA IN THE MU3E EXPERIMENT

MU3E CDB Browser

Global Tags

Filter global tags...

- datav6.3=2025V1test
- datav6.3=2025V2test
- datav6.3=2025V3test
- datav6.3=2025V1**
- datav6.3=2025V0

Tags datav6.3=2025V1

- pixelalignment_datav6.3=2025V1
- fibrequality_datav6.3=2025V0
- tilealignment_datav6.3=2025V0
- pixelefficiency_datav6.3=2025V0**
- tilequality_datav6.3=2025V0
- fibrealignment_datav6.3=2025V0

Payloads pixelefficiency_datav6.3=2025V0 (Total IOVs: 4) (4 payloads)

IOV	Date	Comment	Schema
5700	12/15/2025, 5:57:59 PM	ascii/sectors_eff_runs_05700-05799_chi2p2_0_chi2f15_0_pp25_0_pf5_0_l6.csv	ui_id,i_n[.efficiency]
200	12/10/2025, 4:07:59 PM	pattern	ui_id,i_n[.efficiency]
100	12/10/2025, 4:08:20 PM	all dead	ui_id,i_n[.efficiency]

Detconfigs Summary

Tag	Files	Actions
tdac_files_bu_06_06_tuning_first	109	Download Delete
tdac_files_bu_06_10	109	Download Delete

[Upload Files](#)

Running on mu3edb0

Figure 2: CDB browser illustrating a selection of global tags, the tags making up the global tag `datav6.3=2025V1` and the payloads referenced in the tag `pixelefficiency_datav6.3=2025V0`. Clicking on the payload row opens a popup window showing the decoded payload content.



DATABASES FOR NON-EVENT DATA IN THE MU3E EXPERIMENT

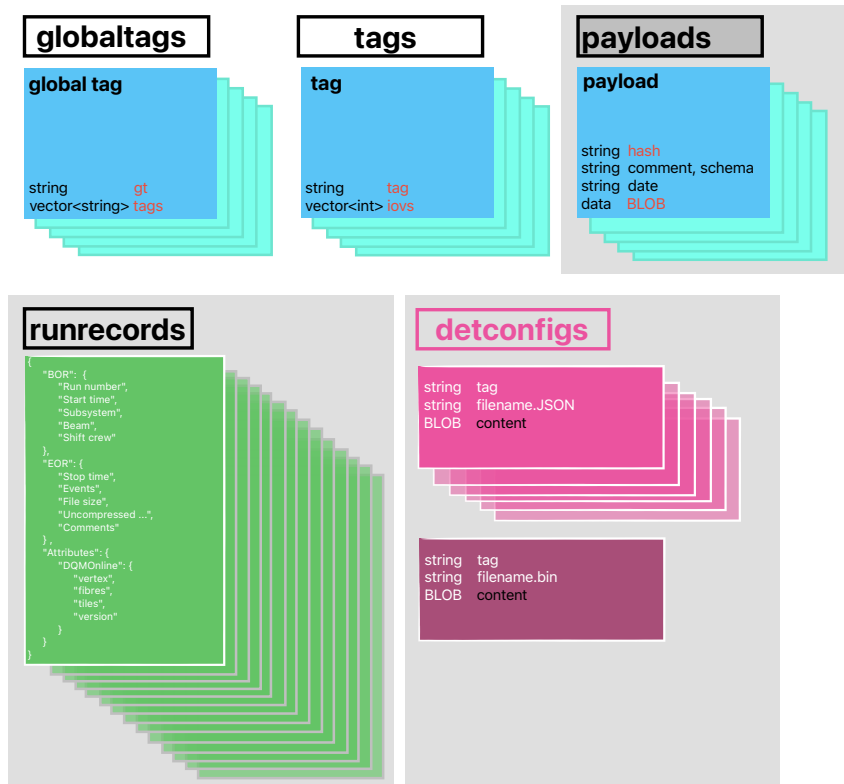


Figure 3: Data model of the conditions database. Currently, the "string" BLOB in the payload is a base64 encoded string, but eventually could be replaced with true binary data.



2 Hardware and System Setup

The databases for non-event data are hosted by one server, `mu3edb0`, located in the racks of the Mu3e DAQ room. The machine has 8 cores (Xeon(R) Silver 4309Y CPU at 2.80GHz) and 128GB of RAM. It has two 10G ethernet interfaces, one with a fixed IP address assigned, `129.129.143.6` with MAC address `ec:e7:a7:11:23:58`, in the normal PSI network (*not* the mu3e network!). The second ethernet interface could provide for possible integration into the mu3e network, if required in the future. There is a user account `mu3e` intended for all system work (via `sudo`). The password for this user is written on the server.

`nginx` is configured to redirect

```
https://mu3edb0.psi.ch/cbd → https://mu3edb0.psi.ch:5050/cbd
https://mu3edb0.psi.ch/rbd → https://mu3edb0.psi.ch:5050/rbd
http://mu3edb0.psi.ch/cbd → http://mu3edb0.psi.ch:5050/cbd
http://mu3edb0.psi.ch/rbd → http://mu3edb0.psi.ch:5050/rbd
https://mu3edb0.psi.ch      → https://mu3edb0.psi.ch:3001
```

The last line is relevant for the partsDB. The relevant configuration files are stored in Ref. [2]

Software RAID (level 1) integrates (1) two 240GB SATAIII SSD disks into the root file system and (2) two 24 TB SASIII HDD disks into two data partitions for the CDB/RDB and the partsDB:

- `/dev/mapper/vg0-lv-0 228422036 16692932 200053048 8% /`
- `/dev/mapper/vg_data-lv_couchdb 2112647088 75730440 1929526084 4% /mnt/data2`
- `/dev/mapper/vg_data-lv_mongodb 21288065024 176686488 21111378536 1% /mnt/data1`

In total 12 SSD/HDD Drive Bays are available, the majority still free for future disk additions.

2.1 CDB and RDB

The user account for all CDB/RDB related work and code is `mu3ecdb`, with the same password as the user `mu3e`. All relevant code for the CDB/RDB node server is located in `/home/mu3ecdb/mu3eancadb1/rest/`.

2.2 PartsDB

The user account for all CDB/RDB related work and code is `mu3epartsdbadmin`. The password for this user is written on the server. All relevant code for the CDB/RDB node server is located in `/home/mu3epartsdbadmin/mu3epartdb/`. **Important:** The PartsDB server running on `mu3edb0` is *not* where you should access the PartsDB. Use the original in Heidelberg. Its intention is to provide for a backup server in case anything happens to the server in Heidelberg (though some finetuning to the reverse proxy is required for proper write and database access).



3 Software architecture and API

This section describes how non-event data are served to Mu3e software and which interfaces exist outside reconstruction code. The logical data model (global tags, tags, IOV lists, payload hashes) is summarized in section 1 and in Ref. [1]; here the focus is on components and machine-facing APIs.

3.1 Architecture

Storage and processes. The primary store is a MONGODB database on the server `mu3edb0` (collections for conditions data, run records, detector configuration file bundles, and optional GridFS buckets for large binary objects). An EXPRESS application (`db1/rest`) connects to the same database and exposes JSON over HTTP. Mu3e C++ code does not talk to MONGODB directly for routine access; it uses a small abstraction layer that can target either the REST service or a directory tree of JSON documents (offline use, tests, or air-gapped copies).

Abstract database access (`cdbAbs`). The class `cdbAbs` defines the interface used by conditions and run-configuration logic. Implementations include:

- `cdbJSON` — reads the same document layout from a local directory (one file per document where appropriate);
- `cdbMongo` — native driver access to a MONGODB instance;
- `cdbRest` — HTTP client (`libcurl`) against the Express routes under the `/cdb` prefix (subsection 3.2).

The interface covers: global tags, per-global-tag tag lists, IOV arrays per tag, payload retrieval by hash, configuration documents by hash, and run records by run number, plus helpers to list run numbers. Comments on global tags and tags are exposed for tooling and the web browser.

Conditions coordinator (`Mu3eConditions`). `Mu3eConditions` is the singleton entry point used in reconstruction and analysis. For a chosen global tag it caches tag names and IOV maps, resolves the correct hash for a given run, and notifies registered calibration objects when the run changes. It also exposes run records and registered configuration payloads (`cfgPayload`) alongside calibration access.

Calibration layer (`calAbs` and derivatives). Each calibration type subclasses `calAbs`. The base class handles caching of `payload` objects, hash-based refresh, and file I/O; derived classes implement `calculate`, BLOB encoding and decoding, and typed accessors. The payload “schema” is therefore defined in code, not in the database engine—as in Ref. [1]. Concrete classes are instantiated via `Mu3eConditions::createClass` and `Mu3eCalFactory`.



Writing and maintenance tools. Payload JSON files and bootstrap inputs are produced by standalone programs (e.g. `cdbPayloadWriter`, `cdbRunPayloadWriter`) and sync utilities (`syncJSON`, `syncMongo`, `syncRDB`, `syncCloud`). These are part of the same source tree as the calibration classes (`db0/cdb2`) and are used to populate or mirror the database; they are not required inside event processing.

Figures. The storage-oriented overview (conditions collections, run records, detector-configuration bundles) is Fig. 3 in section 1. The C++ structure is shown in Fig. 4.

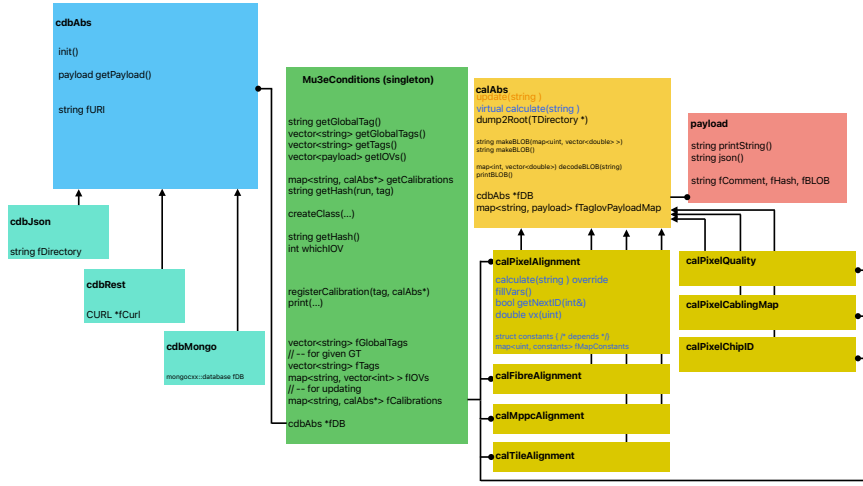


Figure 4: Class organization for Mu3e conditions management and access to database storage and calibration constants (see also Ref. [1]).

3.2 HTTP API

The REST application listens on a configurable port (default 5050) and mounts route modules as follows: `/cdb` conditions and related helpers, `/rdb` run-database pages and run-record ingestion, `/ddb` a simple listing view for detector-configuration documents, and `/dqm` for DQM-related assets. The C++ client `cdbRest` is configured with the *base URL of the CDB router*, i.e. `http(s)://<host>:<port>/cdb`, not the server root.

Core read pattern. The routes consumed by `cdbRest` follow a uniform pattern:

- `GET /cdb/findOne/<collection>/:id` — return one document or Not found. Collections used here include `globaltags` (key: `gt`), `tags` (key: `tag`), `payloads` (key: `hash`), `configs` (key: `cfgHash`), and `runrecords` (key: run number in the URL path, matched against the BOR run-number field).
- `GET /cdb/findAll/<collection>` — list or scan a collection; used for global tags and for the flat list of run numbers (`/cdb/findAll/runNumbers`).



For payloads, large BLOBs may be stored in GridFS; the server merges the grid file into the document as a base64 BLOB field before sending the response, so clients always see the same JSON shape.

Authentication. `cdbRest` sends an API key in an `api-key` header; the key is read from a local file (`api-key.private`) at startup. The exact policy (header name, TLS termination, reverse proxy) is part of the server deployment; the C++ side only requires that the key matches what the proxy or Node process expects.

Additional CDB routes (browser and tools). Beyond the minimal `findOne/findAll` surface, the CDB router exposes helpers used by the web UI and command-line utilities, including:

- `GET /cdb/findTagsByGlobaltag/:globaltag` — tag metadata and IOV arrays for one global tag;
- `GET /cdb/findPayloadsByTag/:tag` — payload summaries (with optional limit) for hashes belonging to a tag;
- `GET /cdb/detconfigTags` and `GET /cdb/findAll/detconfigsSummary` — inventory of detector-configuration tags;
- upload and download routes for detector-configuration file bundles (multipart `/cdb/upload`, `/cdb/uploadMany`, `/cdb/downloadTag`, JSON-oriented `/cdb/uploadJSON`, `/cdb/downloadJSON/...`), and delete-by-tag operations for the `detconfigs` collection.

These routes overlap with small dedicated executables in `db0/cdb2` (e.g. upload/download of detector configuration) that call the same HTTP surface.

Run database API. Run records are read through `cdbAbs::getRunRecord` and listed via `getAllRunNumbers`; on the server side the RDB module (`/rdb`) provides paginated and filtered HTML/JSON views, and `PUT /rdb/runrecords` inserts or replaces a document for one run. Details of run-document schema and DQM linkage belong in the RDB and workflow sections; only the split of URL space is noted here so that “CDB” and “RDB” endpoints are not confused.



4 Calibration Software

This section describes the software used for calibration of the Mu3e detector. The calibration classes are the classes that are used to provide non-event data (calibration data).

4.1 calAbs

The `calAbs` class is the base class for all calibration classes and provides all functionality in connection to the database and `Mu3eConditions`. Everything requiring a detailed understanding of the payload (schema), is implemented in derived classes.

4.2 calEventStuffV2

The `calEventStuffV2` class is used to provide the CDB access to quantities that could just as well be stored in/accessed from the RDB. The primary use case is for indicating frame numbers where DAQ problems started. For the entire “event” and for each subsystem (pixel, fibres, tiles) it provides the following information:

- `startFrame(Good)Data`
- `endFrame(Good)Data`
- `firstFrameWithFEBProblems`

where the “(Good)” label applies only to the subsystem data. The payload schema is

```
"eventdata.(ull_startframedata,  
            ull_endframedata,  
            ull_firstframewithfebproblems),  
pixeldata.(ull_startframegooddata,  
            ull_endframegooddata,  
            ull_firstframewithfebunsortedhitdata),  
tiledata.(ull_startframegooddata,  
            ull_endframegooddata,  
            ull_firstframewithfebunsortedhitdata),  
fibredata.(ull_startframegooddata,  
            ull_endframegooddata,  
            ull_firstframewithfebunsortedhitdata)"
```

The user API provides the obvious getter methods for the above quantities.

4.3 calTileTimeCalibration

The `calTileTimeCalibration` class is used to provide the tile time calibration data, consisting of corrections for

- differential non-linearities (DNL): 32 doubles per channel
- time walk (TW): one double per channel



- time alignment (TA): energy dependent doubles stored as vectors with variable (channel dependent) size. The vector with the energy binning is stored in the payload, after the correction constants.

The payload schema is

```
"ui_id,d_dnl[32],d_ta,i_tw_nbins[,d_tw_ns][,d_tw_energy]"
```

The user API provides the following methods:

- `double getTimeAlignmentOffsetNS(uint32_t id)`
- `double getDNLCorrectedTimeFraction(uint32_t id, int ibin)`
- `double getTimeWalkCorrectionNS(uint32_t id, double energy)`
This function does a lookup of the correct energy bin (initializing the bin search assuming an equidistant binning).
- `vector<double> getTimeWalkCorrectionNS(uint32_t id)`
- `vector<double> getTimeWalkCorrectionEnergy(uint32_t id)`
- `vector<double> getDNLCorrectedTimeFraction(uint32_t id)`

It should be noted that the `energy` argument in `getTimeWalkCorrectionNS` has the same units as the "energy" in the "timewalk_correction" section of the JSON file.



5 Workflows and tools

5.1 Overview

All commands in this section are in `mu3eUtil/cdb` or in `mu3eanca/db0/cdb2`. Make sure that your `PATH` includes the corresponding directories for their executables. While we do provide examples for each workflow, we do not cover all possible aspects—check the source code for more details.

- CDB global tag listing: `cdbSummaryGT` provides summary information about global tags, tags, and payloads. It allows filtering and searching.
- Payload creation: The primary tool for creating and adding payloads to the CDB is `cdbRunPayloadWriter`, a wrapper for `cdbPayloadWriter`.
- Tag creation/IOV insertion: `cdbInsertIovTag` writes (creates or modifies) the tag JSON file.
- Payload inspection: `cdbPrintPayload` provides textual output of the payload.
- CDB mongoDB upload: `syncMongo`
- CDB mongoDB download: `syncJSON`

5.2 Lists for detector element IDs

The list of detector element IDs is stored in the `cdbPayloadWriter` source code. This is a feature. Currently, the following lists are available:

- MuPix11 sensor IDs for 2025 (two-layer VTX), 2026 (two-layer VTX plus central Layer 3), and the complete detector
- Fibres and MPPC IDs for 2025 (two modules) and the complete detector
- Tile IDs for 2025 and the complete detector

These lists are produced, for instance, with invocations like

```
cdbRunPayloadWriter -m createsensorids  
-f ascii/mu3e_alignment_mcidealv6.5.root
```

Most of the methods can also read from alternative input files, e.g., from a CSV or JSON file. (And yes, this is very old-fashioned, but it is a reproducible method and the input files are kept for reference.)

5.3 New tag with new initial payload(s)

A new tag (most often) implies a new payload, which needs to be created and added to the CDB.



5.3.1 Alignment/geometry payload with root alignment input

It is possible that changes in mu3eSim and/or mu3eTriRec require new payloads, e.g., if the numbering scheme changes, if the orientation of the local coordinate system changes, etc. An incomplete subdetector is another case where a new tag plus payload is required.

- All new alignment/geometry payloads

```
cdbRunPayloadWriter -c alignment
-f ascii/mu3e_alignment_mcidealv6.8.root
-a "some good annotation string"
-p ~/data/mu3e/cdb/payloads
-g mcidealv6.9
```

- One new alignment/geometry payloads, for a specific era (defined in cdbPayloadWriter)

```
cdbRunPayloadWriter -c fibrealignment
-m 2025
-f ascii/mu3e_alignment_mcidealv6.5.root
-a "as installed in 2025"
-p ~/data/mu3e/cdb/payloads
-g datav6.5=2025V1
```

This will use the list of IDs in the cdbPayloadWriter configuration to filter the input of the alignment tree such that only the relevant detector elements are included in the payload.

The same approach can be used for an updated alignment/geometry tree for a specific subdetector in a root file, e.g., from detector alignment with tracks or other means (metrology).

5.3.2 New payloads based on many input files in a directory

A new tag for eventStuffV2 provides an examples how to proceed with many input files prepared in a single directory. The input files need to follow specific naming conventions like run06219.mid.lz4.json, pixelqualitylm-run6755.csv, etc.

```
./bin/cdbRunPayloadWriter
-c eventstuffv2
-d ~/data/mu3e/run2025/rawv2
-a "includes skipped header bug for pixels, but nothing on fibres/tiles"
-t datav6.9=2025V0
```

This will produce also the payload for run 1 (with perfect settings). If these payloads are driven by a new class, e.g., calEventStuffV2, all other GTs for this release need to reflect this change. Therefore create, e.g., a new mcidealv6.9 GT with

```
./bin/cdbRunPayloadWriter -c eventstuffv2
-a "ideal tag, for inclusion of skipped header bug for pixels"
-f ascii/eventstuffv2-ideal.json
-t mcidealv6.9
-r 1
```



5.3.3 New payloads from detector input and initial payload

A new tag (most often) implies new payload information from subsystems and a new initial payload (to be created manually). If the new tag is for a partially installed detector, the payloads for run 1 and the subsequent IOVs should match in size.

- First create the payloads from JSON/CSV input files created by the subsystems

```
./bin/cdbRunPayloadWriter -c tiletimecalibration
-t datav6.9=2025V0
-f ~/Downloads/calibration_run03274_config.json
-a "First (placeholder) version. Dead channels w/o constants."
-i 3274
```

- Then create the corresponding payload for run 1 (with the same size)

```
./bin/cdbRunPayloadWriter -m tiletimecalibration-ideal-2025
-f ascii/tiletimecalibration-ideal-2025.json
```

```
./bin/cdbRunPayloadWriter -c tiletimecalibration
-t datav6.9=2025V0
-f ascii/tiletimecalibration-ideal-2025.json
-a "Created (initially) for GT datav6.9=2025V0"
-i 1
-m 2025
```

```
./bin/cdbRunPayloadWriter -m pixelmask-ideal -a "ideal tag, for completeness only"
-t mcideallv6.9 -r 1
```

- Once you have assembled all payloads for the new tag, move the directory with the payloads to the CDB directory and insert the tag with the `cdbInsertIovTag` tool (see next subsection 5.3.4).

5.3.4 Insert new tag with new (initial) payload(s)

The new tag can be inserted with the `cdbInsertIovTag` tool.

```
cdbInsertIovTag -t mppcalignment_datav6.5=2025V1
-j /Users/ursl/data/mu3e/cdb
-d ~/data/mu3e/cdb/payloads/mppcalignment_datav6.5=2025V1
-p mppcalignment_datav6.5=2025V1
```

5.3.5 Upload new tag with payloads to mongoDB

The new tag with payloads can be uploaded to mongoDB using the `syncMongo` tool.

```
syncMongo --dir ~/data/mu3e/cdb
--host $DBHOST --sync tag
--name mppcalignment_datav6.5=2025V1
--deep --del
```



The `-deep` option ensures that all referenced payloads are uploaded as well, and the `-del` option deletes the tag from the mongoDB database, if it already exists there. `$DBHOST` is the host name of the mongoDB server: initially this should be set to `localhost`, and after all checks are done, it should be set to the actual host name, usually `mu3edb0.psi.ch`.

5.3.6 Upload new global tag to mongoDB

The new global tag can be uploaded to mongoDB using the `syncMongo` tool.

```
syncMongo --dir ~/data/mu3e/cdb
--host $DBHOST --sync gt
--name mcrealisticv6.6=2025V1 --del
```

The `-del` option deletes the GT from the mongoDB database, if it already exists there. `$DBHOST` is the host name of the mongoDB server: initially this should be set to `localhost`, and after all checks are done, it should be set to the actual host name, usually `mu3edb0.psi.ch`.

5.3.7 Download GT and tags with payloads from mongoDB

The GT and tags with payloads can be downloaded from mongoDB using the `syncJSON` tool.

```
setenv CDB /Users/urs1/data/mu3e/test-cdb

syncJSON --cdb --dir $CDB --host mu3edb0
-m tag -e mppcalignment_datav6.5=2025V1 --deep

syncJSON --cdb --dir $CDB --host mu3edb0
-m tag -e fibrealignment_datav6.5=2025V1 --deep

syncJSON --cdb --dir $CDB --host mu3edb0
--sync gt --name mcrealisticv6.5=2025V1
```



6 Changes to the CDB code

6.1 mu3eUtil v6.9

- calEventStuffV1 is deprecated.
- calEventStuffV2 providing information on skipped header bug for pixels, but not for fibres of tiles. It is strongly recommended to query the validity of the payload using the new method `isValid(int runNumber)`. The payload does not make sense if its IOV is different from the run number.
- calTileTimeCalibration providing tile time calibration data (first draft, calibration data to be corrected)

6.2 mu3eUtil v6.7

- added small node serve to serve cdbJSON on local machine to browser
- big rewrite for cdbPayloadWriter (uses cdbPayloadWriter)
- added `-n` (no payload reading as this can be slow) to cdbSummaryGT, the ultimate online documentation for GTs, tags, and more in the CDB

6.3 mu3eUtil v6.6

- added cdbSummaryGT as CLI interface to CDB contents, with filtering possibilities etc
- calPixelTimeCalibration is no longer constrained to be 108 chips
- calPixelMask: new calibration class for pixel mask access (normally very different IOVs than pixelquality, hence different tag/payload)
- calPixelQualityLM will include calPixelMask (in case it's present)
- add new method `getPayloadSize()`, returning number of detector elements in payload
- add `downloadDetConfig` (REST api)

6.4 mu3eUtil v6.5

- can manually replace a tag with a new one using the implementation of `replaceTag()`
- add `mu3e::conf.cdb.replaceTags`
- add subdirectories for payload storage (keep backward compatibility)
- api migration (remove unused `gt` from `cdb*` instantiation)
- runblocks also for runrecords
- change default GT and CDB location to correspond to merlin7
- add `eventData.endFrame`



DATABASES FOR NON-EVENT DATA IN THE MU3E EXPERIMENT

- new approach to DB initialization
- utilities for adding/reading comments for GTs and tags
- remove old-style casts, fix compiler warnings



7 Global Tags

A “global tag” (GT) is a collection of tags. Global tags provide a complete and coherent set of tags for the analysis of detector data and MC samples (both in an idealized and a realistic setting). A tag is a name and a collection of IOVs. An IOV is an interval of validity, with one payload per IOV. A payload is a collection of calibration constants.

GTs for detector data have **data** and the running period (e.g. 2025, but this can change to accomodate “eras”, e.g. 2026A, 2026B, etc.) in their name and contain all calibration and geometry constants required for an optimal analysis of the detector data. For unstable detector conditions, the IOV list will be quite long.

GTs for MC samples have **mc** in their name and come in two variants:

- **mcideal** for the ideal MC setup. This by convention means the full detector as present in the standard simulation setup.
- **mcrealistic** for non-ideal MC setups. "Non-ideal" means that the simulation setup is missing parts of the detector, to properly describe the detector as installed. In addition, these GTs should contain smeared geometry/alignment information to better describe the detector performance to match the performance of the real detector.

For instance, the information in **pixelqualitylm** will be very different for the two variants: The tag **pixelqualitylm_mcideal** will contain a perfect pixel quality payload (with one IOV), while the tag **pixelqualitylm_mcrealistic** will contain the holes and noisy pixels as present in the real detector with many IOVs.

Similarly, the alignment/geometry information will be different in the three cases: For data, this is the alignment/geometry information obtained with dedicated alignment procedures for the real detector. For **mcideal**, this is the alignment/geometry information from the ideal MC setup. For MC realistic, this is the alignment/geometry information that makes the MC simulation more realistic (this could well be a data tag for the alignment/geometry information, but not for the pixeltiming calibration constants, because here the perfect MC timing must be smeared to describe the data, while the data have to be corrected for timewalk, etc). In the absence of dedicated alignment procedures for the real detector, the alignment/geometry information for **mcrealistic** are taken from the ideal MC setup (filtering out the parts of the detector that are not present in the real detector).

The following considerations should be taken into account when choosing the global tag:

- The global tag must be compatible with the software version. For example, if the software version is v6.3.x, the global tag should be **datav6.3=2025V1** since GTs with a later release in their name may contain tags that are incompatible with the software version. Furthermore changes to reconstruction code may require new tags (e.g. numbering schemes, detector element orientation, etc.) and using the wrong software version will results in incorrect results.



It is often permissible to use a GT with an earlier release in its name, if the tags are compatible with the software version (if only changes to the reconstruction or analysis code have been included in the code base).

- The global tag should be compatible with the data taking period. It is possible that not all tags are produced for all data taking periods.
- The global tag should be compatible with the detector configuration. You should not use a three-layer GT when analyzing two-layer data.
- You should never use a GT with **test** in its names, except for testing purposes.

Table 1: Data global tags for mu3e tagged releases v6.X

GT	Description
datav6.3=2025V0 Do not use!	November 2025. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmons). Alignment from MC. pixelQualityLM payloads are incorrect!
datav6.3=2025V1 First GT with improved pixel alignment.	December 2025, for BVR2026. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmons, with proper B field). VTX alignment based on metrology and cosmons data (DF and MS, respectively). pixelQualityLM payloads are incorrect!
datav6.5=2025V0 Do not use!	March 2026. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmons, with proper B field). VTX alignment based on metrology and cosmons data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file. pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, to be corrected)
datav6.5=2025V1 First GT with (1) correct LVDS error rate from MIDAS meta data, (2) pixelEfficiency data and (3) fibres alignment information as installed.	June 2026. Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and tiles (beam and cosmons, with proper B field). VTX alignment based on metrology and cosmons data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file. pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected)

Continued on next page



Table 1 – continued from previous page

GT	Description
datav6.6=2025V0 Do not use!	March 2026. Requires at least CDB-v6.6pre for mu3eUtil. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmits, with proper B field). VTX alignment based on metrology and cosmits data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs ≥ 6865 , see pixelmask tag comment). pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected)
datav6.6=2025V1 First v6.6 GT with proper fibres/mppcs alignment information as installed.	June 2026. Requires at least CDB-v6.6pre for mu3eUtil. Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and tiles (beam and cosmits, with proper B field). VTX alignment based on metrology and cosmits data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs ≥ 6865 , see pixelmask tag comment). pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected)
datav6.9=2025V0 First GT with (1) eventStuffV2 including “skipped header bug” information and (2) tileTimeCalibration.	June 2026. Requires at least CDB-v6.9pre for mu3eUtil. Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and tiles (beam and cosmits, with proper B field). VTX alignment based on metrology and cosmits data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs ≥ 6865 , see pixelmask tag comment). pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected). tileTimeCalibration payloads are placeholders.

In the following, the GTs are described in more detail.



7.1 **datav6.3=2025V0**

Do not use this tag, it was superseded by tag **datav6.3=2025V1** in December 2025.

Problems with this tag:

- The pixel quality LM payloads are incorrect since the MIDAS metadata was not correctly parsed.
- The pixel alignment is based on the ideal MC alignment. Also true for the fibre and tile alignment information.

Description of the tag:

GT: **datav6.3=2025V0**

tags: 11

comment: Do not use! November 2025. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics). Alignment from MC. pixelQualityLM payloads are incorrect!

- 1) **pixelalignment_datav6.3=2025V0**
IOV length: 1, comment: VTX only, based on MC
- 2) **fibrealignment_datav6.3=2025V0**
IOV length: 1,
- 3) **tilealignment_datav6.3=2025V0**
IOV length: 1
- 4) **mppcalignment_datav6.3=2025V0**
IOV length: 1
- 5) **pixelqualitylm_datav6.3=2025V0**
IOV length: 3550
- 6) **detsetupv1_datav6.3=2025V0**
IOV length: 3
- 7) **pixeltimecalibration_datav6.3=2025V0**
IOV length: 1
- 8) **tilequality_datav6.3=2025V0**
IOV length: 13
- 9) **fibrequality_datav6.3=2025V0**
IOV length: 99
- 10) **pixelefficiency_datav6.3=2025V0**
IOV length: 4
- 11) **eventstuffv1_datav6.3=2025V1**
IOV length: 3552



7.2 datav6.3=2025V1

This tag is the best tag for 2025 data analysis with mu3e v6.3.x releases, though the pixelQualityLM payloads are not as good as in later GTs (datav6.5=2025VX, etc.). First GT with improved pixel alignment. DF = David Fritz, MS = Mikio Sakurai.

Description of the tag:

GT: datav6.3=2025V1

tags: 11

comment: December 2025, for BVR2026. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads are incorrect!

- 1) pixelalignment_datav6.3=2025V1
IOV length: 1
comment: VTX only, based on metrology and cosmics data (DF and MS, respectively).
- 2) fibrealignment_datav6.3=2025V0
IOV length: 1
- 3) tilealignment_datav6.3=2025V0
IOV length: 1
- 4) mppcalignment_datav6.3=2025V0
IOV length: 1
- 5) pixelqualitylm_datav6.3=2025V0
IOV length: 3550
- 6) detsetupv1_datav6.3=2025V0
IOV length: 3
- 7) pixeltimecalibration_datav6.3=2025V0
IOV length: 1
- 8) tilequality_datav6.3=2025V0
IOV length: 13
- 9) fibrequality_datav6.3=2025V0
IOV length: 99
- 10) pixelefficiency_datav6.3=2025V0
IOV length: 4
- 11) eventstuffv1_datav6.3=2025V1
IOV length: 3552



7.3 `datav6.5=2025V0`

Do not use this tag, it was superseded by tag `datav6.5=2025V1` in June 2026.
TCS = Thomas Senger. Problems with this tag:

- The fibre/mppc alignment is based on v6.3 and does not incorporate the changed orientation, etc..

Description of the tag:

GT: `datav6.5=2025V0`

tags: 11

comment: March 2026. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). `pixelQualityLM` payloads correctly include LVDS errors from MIDAS meta data file. `pixelEfficiency` with reduced iov list: 1 (perfect), 5700 (TCS, to be corrected)

- 1) `pixelalignment_datav6.3=2025V1`
IOV length: 1
comment: VTX only, based on metrology and cosmics data (DF and MS, respectively).
- 2) `fibrealignment_datav6.3=2025V0`
IOV length: 1
- 3) `tilealignment_datav6.3=2025V0`
IOV length: 1
- 4) `mppcalignment_datav6.3=2025V0`
IOV length: 1
- 5) `pixelqualitylm_datav6.5=2025V0`
IOV length: 3330
comment: Correctly includes the LVDS errors from the midas meta data file
- 6) `detsetupv1_datav6.3=2025V0`
IOV length: 3
- 7) `pixeltimecalibration_datav6.3=2025V0`
IOV length: 1
- 8) `tilequality_datav6.3=2025V0`
IOV length: 13
- 9) `ibrequality_datav6.3=2025V0`
IOV length: 99
- 10) `pixelefficiency_datav6.5=2025V0`
IOV length: 2
comment: perfect efficiencies starting from run 1 (no experimental numbers available), from run 5700 onwards numbers from TCS (likely underestimating pixel efficiency, with known bias).
- 11) `eventstuffv1_datav6.3=2025V1`
IOV length: 3552



7.4 datav6.5=2025V1

This tag is the correct tag for 2025 data analysis with mu3e v6.5.x releases. First GT with (1) correct LVDS error rate from MIDAS meta data, (2) pixelEfficiency data and (3) fibres alignment information as installed.

Description of the tag:

GT: datav6.5=2025V1

tags: 11

comment: June 2026. Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file. pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, to be corrected)

- 1) pixelalignment_datav6.3=2025V1
IOV length: 1
comment: VTX only, based on metrology and cosmics data (DF and MS, respectively)
- 2) fibrealignment_datav6.5=2025V1
IOV length: 1
- 3) tilealignment_datav6.3=2025V0
IOV length: 1
comment: Database entry inserted
- 4) mppcalignment_datav6.5=2025V1
IOV length: 1
- 5) pixelqualitylm_datav6.5=2025V0
IOV length: 3330
comment: Correctly includes the LVDS errors from the midas meta data file
- 6) detsetupv1_datav6.3=2025V0
IOV length: 3
comment: Database entry inserted
- 7) pixeltimecalibration_datav6.3=2025V0
IOV length: 1
comment: Database entry inserted
- 8) tilequality_datav6.3=2025V0
IOV length: 13
comment: Database entry inserted
- 9) fibrequality_datav6.3=2025V0
IOV length: 99
comment: Database entry inserted
- 10) pixelefficiency_datav6.5=2025V0
IOV length: 2
comment: perfect efficiencies starting from run 1 (no experimental numbers available), from run 5700 onwards numbers from TCS (likely underestimating pixel efficiency, with known bias).
- 11) eventstuffv1_datav6.3=2025V1
IOV length: 3552



DATABASES FOR NON-EVENT DATA IN THE MU3E EXPERIMENT

comment: Database entry inserted



7.5 datav6.6=2025V0

Do not use this tag, it was superseded by tag `datav6.6=2025V1` in June 2026.
Problems with this tag:

- The fibre/mppcs alignment is based on v6.3 and does not incorporate the changed orientation, etc..

Description of the tag:

GT: `datav6.6=2025V0`

tags: 12

comment: March 2026. Requires at least CDB-v6.6pre for mu3eUtil.
Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). `pixelQualityLM` payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs ≥ 6865 , see `pixelmask` tag comment). `pixelEfficiency` with reduced iov list: 1 (perfect), 5700 (TCS, to be corrected)

- 1) `pixelalignment_datav6.3=2025V1`
IOV length: 1
comment: VTX only, based on metrology and cosmics data (DF and MS, respectively).
- 2) `fibrealignment_datav6.3=2025V0`
IOV length: 1
comment: Database entry inserted
- 3) `tilealignment_datav6.3=2025V0`
IOV length: 1
comment: Database entry inserted
- 4) `mppcalignment_datav6.3=2025V0`
IOV length: 1
comment: Database entry inserted
- 5) `pixelqualitylm_datav6.5=2025V0`
IOV length: 3330
comment: Correctly includes the LVDS errors from the midas meta data file
- 6) `pixelmask_datav6.6=2025V0`
IOV length: 3
comment: pixel masks: Tune 1 and Tune 2. No proper mask files available for cosmic runs (additional masking of 20 rows/cols around edges required - DIY)
- 7) `detsetupv1_datav6.3=2025V0`
IOV length: 3
comment: Database entry inserted
- 8) `pixeltimecalibration_datav6.3=2025V0`
IOV length: 1
comment: Database entry inserted
- 9) `tilequality_datav6.3=2025V0`



DATABASES FOR NON-EVENT DATA IN THE MU3E EXPERIMENT

```
IOV length: 13
comment: Database entry inserted
10) fibrequality_datav6.3=2025V0
IOV length: 99
comment: Database entry inserted
11) pixelefficiency_datav6.5=2025V0
IOV length: 2
comment: perfect efficiencies starting from run 1 (no experimental
        numbers available), from run 5700 onwards numbers from TCS
        (likely underestimating pixel efficiency, with known bias).
12) eventstuffv1_datav6.3=2025V1
IOV length: 3552
comment: Database entry inserted
```



7.6 datav6.6=2025V1

This tag is the correct tag for 2025 data analysis with mu3e v6.6.x releases. First v6.6 GT with proper fibres/mppcs alignment information as installed.

Description of the tag:

```
GT: datav6.6=2025V1
tags: 12
comment: June 2026. Requires at least CDB-v6.6pre for mu3eUtil.
        Covers entire 2025 datataking period with two-layer VTX and
        installed fibres/mppcs (thanks to CX) and tiles (beam and
        cosmics, with proper B field). VTX alignment based on
        metrology and cosmics data (DF and MS, respectively).
        pixelQualityLM payloads correctly include LVDS errors from
        MIDAS meta data file and includes the pixel mask file
        information (except for cosmic runs >= 6865, see pixelmask
        tag comment). pixelEfficiency with reduced iov list: 1
        (perfect), 5700 (TCS, to be corrected)

1) pixelalignment_datav6.3=2025V1
   IOV length: 1
   comment: VTX only, based on metrology and cosmics data (DF and MS,
           respectively)
2) fibrealignment_datav6.5=2025V1
   IOV length: 1
3) tilealignment_datav6.3=2025V0
   IOV length: 1
   comment: Database entry inserted
4) mppcalignment_datav6.5=2025V1
   IOV length: 1
5) pixelqualitylm_datav6.5=2025V0
   IOV length: 3330
   comment: Correctly includes the LVDS errors from the midas meta data
           file
6) pixelmask_datav6.6=2025V0
   IOV length: 3
   comment: pixel masks: Tune 1 and Tune 2. No proper mask files
           available for cosmic runs (additional masking of 20
           rows/cols around edges required - DIY)
7) detsetupv1_datav6.3=2025V0
   IOV length: 3
   comment: Database entry inserted
8) pixeltimecalibration_datav6.3=2025V0
   IOV length: 1
   comment: Database entry inserted
9) tilequality_datav6.3=2025V0
   IOV length: 13
   comment: Database entry inserted
10) fibrequality_datav6.3=2025V0
    IOV length: 99
    comment: Database entry inserted
```



DATABASES FOR NON-EVENT DATA IN THE MU3E EXPERIMENT

- 11) `pixelefficiency_datav6.5=2025V0`
IOV length: 2
comment: perfect efficiencies starting from run 1 (no experimental numbers available), from run 5700 onwards numbers from TCS (likely underestimating pixel efficiency, with known bias).
- 12) `eventstuffv1_datav6.3=2025V1`
IOV length: 3552
comment: Database entry inserted



7.7 datav6.9=2025V0

This tag is the correct tag for 2025 data analysis with mu3e v6.9.x releases. It is the first GT with (1) eventStuffV2 including “skipped header bug” information and (2) tileTimeCalibration. The naming convention for the alignment tags for all subdetectors except the VTX changed consistently to mcidealv6.9=2025 to indicate that this corresponds to unsmeared geometry/alignment information for the installed detector components.

Description of the tag:

GT: datav6.9=2025V0

tags: 13

comment: July 2026. Requires at least CDB-v6.9pre for mu3eUtil.
Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and installed tiles. VTX alignment based on metrology and cosmics data (DF and MS, respectively). eventStuffV2 contains skipped header bug information. pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs ≥ 6865 , see pixelmask tag comment). pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected). eventStuffV2 payloads complete (except for crashing jobs). tileTimingCalibration payloads are placeholders so far.

- 1) detsetupv1_datav6.3=2025V0
IOV length: 3
comment: Database entry inserted
- 2) eventstuffv2_datav6.9=2025V0
IOV length: 6035
- 3) fibrealignment_mcidealv6.9=2025
IOV length: 1
detector units: 754
- 4) fibrequality_datav6.3=2025V0
IOV length: 99
detector units: 96
comment: Database entry inserted
- 5) mppcalignment_mcidealv6.9=2025
IOV length: 1
detector units: 4
- 6) pixelalignment_datav6.3=2025V1
IOV length: 1
detector units: 108
comment: VTX only, based on metrology and cosmics data (DF and MS, respectively)
- 7) pixelefficiency_datav6.5=2025V0
IOV length: 2
detector units: 108
comment: perfect efficiencies starting from run 1 (no experimental numbers available), from run 5700 onwards numbers from TCS (likely underestimating pixel efficiency, with known bias).



DATABASES FOR NON-EVENT DATA IN THE MU3E EXPERIMENT

- 8) pixelmask_datav6.6=2025V0
IOV length: 3
detector units: 108
comment: pixel masks: Tune 1 and Tune 2. No proper mask files
available for cosmic runs (additional masking of 20
rows/cols around edges required - DIY)
- 9) pixelqualitylm_datav6.5=2025V0
IOV length: 3330
detector units: 108
comment: Correctly includes the LVDS errors from the midas meta data
file
- 10) pixeltimecalibration_datav6.3=2025V0
IOV length: 1
detector units: 108
comment: Database entry inserted
- 11) tilealignment_mcidealv6.9=2025
IOV length: 1
detector units: 1248
- 12) tiletimecalibration_datav6.9=2025V0
IOV length: 2
detector units: 1248
- 13) tilequality_datav6.3=2025V0
IOV length: 13
detector units: 5824
comment: Database entry inserted



8 Tags

8.1 Alignment tags

Alignment⁴ tags differ from other calibration tags in the sense that contain absolute coordinates, while other calibration tags contain relative corrections. Pixeltiming calibration constants for **data** should correct as much as possible for imperfect timing measurements (timewalk, biases, etc.), while for **mcrealistic** they should be smear the perfect MC timing to match the data. Correspondingly, the alignment/geometry information for **mcrealistic** can be taken from the corresponding **data** information, but this is not possible for other calibration constants.

⁴Instead of alignment tags we should use the term “geometry” tags, but since this naming scheme has been historically established, it will not be changed.



References

- [1] Urs Langenegger, A conditions database for the mu3e experiment, <https://github.com/ursl/mu3eanca/blob/master/db0/cdb2/slides/Mu3e-Note-0099-ConditionsDatabase.pdf>, 2023.
- [2] Urs Langenegger, `nginx-proxy-common.conf` and `nginx.conf`, <https://github.com/ursl/mu3eanca/tree/master/db2>, 2026.